Smart Security System Technical Report

By Mariam Yahia & Mahmoud Hany

1. Introduction

Physical security remains a critical concern in residential, commercial, and institutional environments. Traditional mechanical locks and standalone alarm systems often fail to provide real-time monitoring, remote control, and adaptive intelligence against theft or intrusion. With the increasing prevalence of Internet of Things (IoT) technologies, it is now feasible to design integrated systems that combine sensors, microcontrollers, cloud services, and mobile applications to deliver robust, scalable, and user-friendly security solutions.

This project aims to develop a complete IoT-based smart security system that detects unauthorized access attempts, provides real-time alerts, and enables remote control of locking mechanisms. The objectives are:

- To design a low-cost yet reliable hardware platform for intrusion detection.
- To integrate firmware logic that ensures accurate sensing and secure actuation.
- To establish a cloud backend for real-time synchronization and data storage.
- To develop a mobile application for monitoring, control, and notifications.
- To demonstrate system scalability and applicability in diverse security scenarios.

2. System Overview

The proposed system consists of the following major components:

- **ESP32 hardware unit**: Serves as the central controller.
- **Sensors (MPU6050 vibration sensor and ultrasonic sensor)**: Detect tampering and proximity-based intrusion.
- **Locking mechanism (relay + solenoid lock)**: Provides physical access control.
- **Cloud backend (Firebase Realtime Database)**: Enables secure data synchronization.
- **Flutter mobile application**: Provides user interface for monitoring and control.

Conceptual Data Flow

1. Sensors detect vibration or proximity events.
2. ESP32 processes sensor data, applies thresholds, and triggers alarms or lock actions.
3. ESP32 updates Firebase with event logs and system status.
4. Flutter application retrieves updates in real time, displays alarms, and allows user intervention.

5. User commands (lock/unlock) are sent back through Firebase to ESP32 for actuation.

3. Hardware Design and Justification

ESP32

- **Reason for selection**: Dual-core processor, integrated Wi-Fi, and Bluetooth make it ideal for IoT applications.
- **Problem solved**: Provides both computation and connectivity in a single low-cost unit.
- **Integration**: Acts as the hub, interfacing with sensors, relay, RTC, OLED, and Firebase.
- **Electrical considerations**: Operates at 3.3V logic; requires level shifting for certain peripherals.

MPU6050 (Vibration Sensor)

- **Reason for selection**: Combines accelerometer and gyroscope for precise motion detection.
- **Problem solved**: Detects tampering attempts such as forced entry or lock manipulation.
- **Integration**: Communicates with ESP32 via I2C bus.
- **Electrical considerations**: Requires stable 3.3V supply and pull-up resistors on I2C lines.

Ultrasonic Sensor

- **Reason for selection**: Reliable distance measurement for detecting intruders near the door.
- **Problem solved**: Provides proximity-based intrusion detection.
- **Integration**: Trigger and echo pins connected to ESP32 GPIOs.
- **Electrical considerations**: Operates at 5V; requires voltage divider for echo pin to ESP32.

Relay + Solenoid Lock

- **Reason for selection**: Provides robust mechanical locking controlled electronically.
- **Problem solved**: Enables remote lock/unlock functionality.
- **Integration**: Relay driven by ESP32 GPIO; solenoid powered by external supply.
- **Electrical considerations**: Requires flyback diode protection and isolation for safety.

RTC (Real-Time Clock)

- **Reason for selection**: Maintains accurate timestamps independent of ESP32 resets.
- **Problem solved**: Provides reliable timekeeping for event logging.
- **Integration**: Communicates via I2C bus.
- **Electrical considerations**: Backup coin cell battery ensures persistence.

OLED Display

- **Reason for selection**: Compact, low-power display for local status visualization.
- **Problem solved**: Provides immediate feedback without requiring mobile app.
- **Integration**: I2C interface with ESP32.
- **Electrical considerations**: Operates at 3.3V; requires proper initialization sequence.

4. Firmware Design (ESP32)

- **Logic Flow**:
  - Initialize peripherals (sensors, RTC, OLED, Wi-Fi).
  - Continuously poll MPU6050 and ultrasonic sensor.
  - Compare readings against predefined thresholds.
  - Trigger alarm and update Firebase if intrusion detected.
  - Listen for Firebase commands to actuate relay/lock.
- **Alarm Thresholds**:
  - MPU6050 vibration threshold tuned to detect tampering but avoid false alarms from minor disturbances.
  - Ultrasonic distance threshold set to detect human presence within 1–2 meters.
- **Locking Mechanism Logic**:
  - Relay energized to unlock; de-energized to lock.
  - Safety consideration: Default state is locked to prevent accidental access.
- **Timestamp Handling**:
  - RTC provides accurate event timestamps.
  - ESP32 logs events with time metadata in Firebase.
- **Alarm Latching**:
  - Once triggered, alarms remain latched until acknowledged via mobile app.
  - Prevents loss of critical alerts due to transient sensor readings.

5. Cloud Architecture (Firebase)

- **Reason for Selection**: Firebase Realtime Database offers low-latency synchronization, scalability, and cross-platform support.
- **Database Structure**:
  - `/status/lock` → current lock state.
  - `/status/alarm` → alarm flag.
  - `/logs/` → timestamped intrusion events.

- **Synchronization**: ESP32 pushes updates; Flutter app listens for changes in real time.
- **Justification**: Firebase eliminates need for custom server, ensuring reliability and scalability.

6. Mobile Application Design (Flutter)

- **Reason for Selection**: Flutter provides cross-platform development with a single codebase.
- **Architecture**:
  - State management via Provider/Bloc for real-time updates.
  - Firebase SDK integrated for database synchronization.
- **Features**:
  - **Monitoring Dashboard**: Displays sensor status, lock state, and alarms.
  - **Manual Lock/Unlock Controls**: Secure remote access control.
  - **Alarm History Logging**: View past intrusion attempts with timestamps.
  - **Push Notifications**: Immediate alerts on intrusion detection.
- **UX/UI Justification**: Minimalist design ensures clarity under stress conditions; intuitive controls reduce user error.

7. System Integration and Data Flow

End-to-End Operation:

1. Sensor detects vibration or proximity.
2. ESP32 processes data and triggers alarm.
3. ESP32 updates Firebase with alarm status and logs.
4. Flutter app receives updates instantly.
5. User receives push notification and can issue lock/unlock commands.
6. ESP32 executes user command and updates Firebase accordingly.

Synchronization and Reliability:

- Firebase ensures low-latency updates (<200 ms typical).
- ESP32 firmware includes retry logic for network failures.
- RTC ensures event logs remain consistent even during connectivity loss.

8. Security, Reliability, and Limitations

- **System Safety**: Relay isolation prevents electrical hazards; solenoid lock defaults to locked state.
- **Fault Handling**: Firmware includes watchdog timers to recover from hangs.
- **Limitations**:
  - Sensors may produce false alarms due to environmental noise.

    - Wi-Fi connectivity required; system performance degrades without stable internet.
    - Power outages disable solenoid lock unless backup battery provided.

9. Applications and Use Cases

- **Home Security**: Protect doors, windows, and entry points.
- **Office and Lab Protection**: Secure sensitive equipment and documents.
- **Locker and Cabinet Security**: Prevent unauthorized access to storage units.
- **Smart Doors and Restricted Areas**: Control access in institutional facilities.
- **Temporary Deployment**: Portable units for events or field operations.

Future Expansion:

- Integration with smart home ecosystems (Alexa, Google Home).
- Commercial scalability for apartment complexes and offices.
- Addition of biometric authentication for multi-factor security.

10. Conclusion

This IoT-based smart security system demonstrates a robust integration of hardware, firmware, cloud, and mobile application components. Each design choice—from ESP32 selection to Firebase synchronization—was made to balance cost, reliability, and scalability. The system provides real-time intrusion detection, secure remote control, and comprehensive logging, making it suitable for diverse applications ranging from homes to laboratories. With future expansions, the system can evolve into a fully integrated smart security platform, adaptable to both personal and commercial environments.